

软件构造

第一章

程序和软件的区别？

- **程序**是为了解决某个特定问题而用程序设计语言描述的适合计算机处理的语句序列
- **软件=程序+数据+文档**(描述软件、开发过程及其管理、操作使用的有关文档)
- 程序是软件的核心组成部分，但软件不只是程序。程序解决“如何实现”，软件解决“如何长期可靠地被使用”。

软件维护的类型？

- 软件交付使用后，为了改正错误、适应环境变化或改善性能而对软件进行的修改活动
- **改正性维护**：修复软件在运行中发现的错误(bug)
- **适应性维护**：为适应外部环境变化而对软件进行修改
- **完善性维护**：在软件正常运行的前提下，为满足新需求或提升性能而进行的修改
- **预防性维护**：为防止未来可能出现的问题，对软件进行的改进

敏捷开发的核心价值：

敏捷开发是一种以人为核心、需求持续变化、以迭代和增量方式进行的软件开发方法，强调快速响应需求变化，通过持续交付可工作的软件来提高开发效率和软件质量。

- **个体和互动**胜过流程和工具
- **工作的软件**胜过详尽的文档
- **客户合作**胜过合同谈判
- **响应变化**胜过遵循计划

软件构造的含义：

指的是通过编码、验证、单元测试、集成测试和调试的组合，详细地创建可工作的、有意义的软件

集成化开发环境的特点：

- **集成化**：将程序开发所需的多种工具集成在一个统一环境中
- **可视化**：提供图形化用户界面
- **自动化**：支持自动编译、自动构建、自动测试

第二章

模块化的原则：高内聚、低耦合

- 一个模块内部各元素之间联系尽可能紧密，共同完成单一明确的功能
- 模块之间的相互依赖关系尽可能少、尽可能简单
- 五个质量属性：模块可分解性、模块可组装性、模块可理解性、模块连续性、模块保护性
- **内聚**由低到高：偶然内聚->逻辑内聚->时间内聚->过程内聚->通信内聚->顺序内聚->功能内聚
- **耦合**由高到低：内容耦合->公共耦合->外部耦合->控制耦合->标记耦合->数据耦合->非直接耦合

测试覆盖与黑盒测试

- **测试用例**是为验证软件是否满足特定需求而设计的一组输入、运行条件和通过或失败准则(预期结果)
- **测试覆盖**指的是测试所包含的软件的特征、元素、成分等方面的程度或范围。
 - 功能覆盖、输入域覆盖、输出域覆盖、函数交互覆盖、代码执行覆盖和代码结构性覆盖
- **黑盒测试**：**不考虑内部实现，仅根据需求和功能说明设计测试用例。**不考虑程序内部结构和内部特性，以用户的角度、从输入数据与输出数据的对应关系角度测试软件

基于等价类划分的测试：

将输入数据划分为若干等价类，并从每个等价类中选取代表值进行测试

- 有效等价类：合法、符合需求的输入数据集合
- 无效等价类：不合法、不符合需求的输入数据集合
- 等价类划分的一般步骤：确定输入条件、划分有效等价类、划分无效等价类、为每个等价类选取一个代表值、设计测试用例
- 优点：用例数量少、设计简单高效
- 缺点：容易忽略边界错误

基于边界值分析的测试：

重点测试输入或输出范围边界及其附近值

- 边界值分析的一般步骤：确定输入或输出范围、确定边界值、选取边界及其附近值、设计测试用例
- 优点：能有效发现边界错误、测试针对性强
- 缺点：不能覆盖所有内部情况

某系统要求输入一个整数 n ，取值范围为 $1 \sim 100$ ，请使用等价类划分和边界值分析设计测试用例。

- 输入类型：整数 合法范围： $1 \leq n \leq 100$ 只有一个输入

- 划分等价类：
 - 有效等价类 E1 $1 \leq n \leq 100$
 - 无效等价类 E2 $n < 1$ E3 $n > 100$ E4 n 不是整数
- 找边界值：合法区间是[1,100]
 - 边界点是下边界：1，上边界：100
 - 附近值 0、1、2 99、100、101
- 选代表值
 - 等价类代表值 E1 50 E2 0 E3 101 E4 3.5
 - 边界值代表 0、1、2 99、100、101
- 整理成测试用例

编号	输入	预期结果
TC1	50	输入合法
TC2	0	输入非法
TC3	101	输入非法
TC4	3.5	输入非法
TC5	1	输入合法
TC6	100	输入合法
TC7	2	输入合法
TC8	99	输入合法

第三章

抽象：

是处理复杂问题的手段，分离对象特性、限制对其当前环境的关注，从众多事物中抽取出共同的、本质性的特征，舍弃非本质特征。对一个过程或一件事物的某些细节有目的地隐藏

- 在东西周围画白盒子，识别出它做什么、不做什么
- 过程抽象：使用函数或方法时知道它能做什么，但不知道它是如何完成的
- 数据抽象：将一个数据类型的特性(值及其运算)与其实现分离

封装：

①把描述一个事物的性质和行为结合在一起成为构件，对外构成该事物的一个界限，使构件能集中而且完整地描述具体的事物，体现了事物的相对独立性②信息屏蔽，即外界不能直接存取构件的内部信息，只能使用提供的数据操作，使用者不必知道操作的内部实现细节，信息隐藏强制封装

- 把事物的性质和行为结合为构件，对外隐藏内部细节，仅提供有限操作接口

- 在东西周围画黑盒子，明确某事能完成，但不知道是怎样做到的

面向对象设计原则 这里特别重要！！理解深刻！

- **单一职责原则(SRP)**
 - 一个类应该只有一个引起它变化的原因
- **开闭原则(OCP) 最重要**
 - 软件实体应对扩展开放，对修改关闭
 - 通过抽象和多态实现扩展，增加功能时尽量不改原代码
- **里氏代换原则(LSP)**
 - 子类型必须能够替换其基类型而不影响程序正确性
 - 继承要“语义一致”
- **接口隔离原则(ISP)**
 - 不应强迫客户端依赖它不需要的方法
 - 接口要小而专，避免“胖接口” 通过细化接口降低依赖
- **依赖倒转原则(DIP)**
 - 高层模块不依赖低层模块，二者都应该依赖抽象
 - 抽象不依赖实现，实现依赖抽象
 - 高层模块定义接口，低层模块负责实现
- **合成复用原则(CARP)**
 - 尽量使用组合/聚合关系，而不使用继承
- **最少知识原则(PLK)**
 - 尽量减少类之间的依赖关系

类图

在 UML 中，类用一个带有类名、可选属性和操作的矩形表示（参考图 3.2）。分隔线用来分离类名、属性和操作。类名在矩形的最上方，随后是属性，最后是操作。约定具体类的名称正常书写，抽象类在类名加上 **abstract** 前缀，或用斜体书写。Java 等语言的接口也可以用类符号表示，接口名加前缀 **interface**，属性空着。属性类似程序语言中的声明——可见性、变量名、类型、初值，用前缀表示可见性：“+”表示公用的，“-”表示私有的，“#”表示保护的。静态变量或常量用大写字母的标识符，成员变量允许有初始值。操作用签名表示——可见性、返回类型、操作名称、参数及类型，可见性符号与属性的相同。有时，为了表示程序中类之间的关系，可以只用一个标示类名的矩形表示类（如图 3.3 所示）。

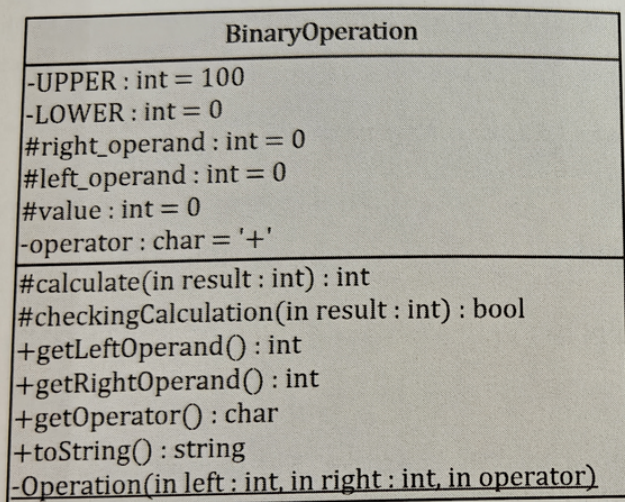


图 3.2 BinaryOperation 的 UML 类图

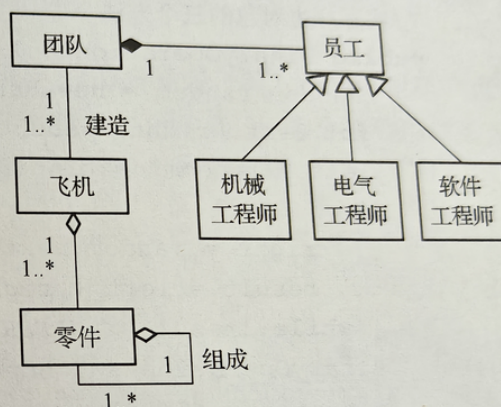
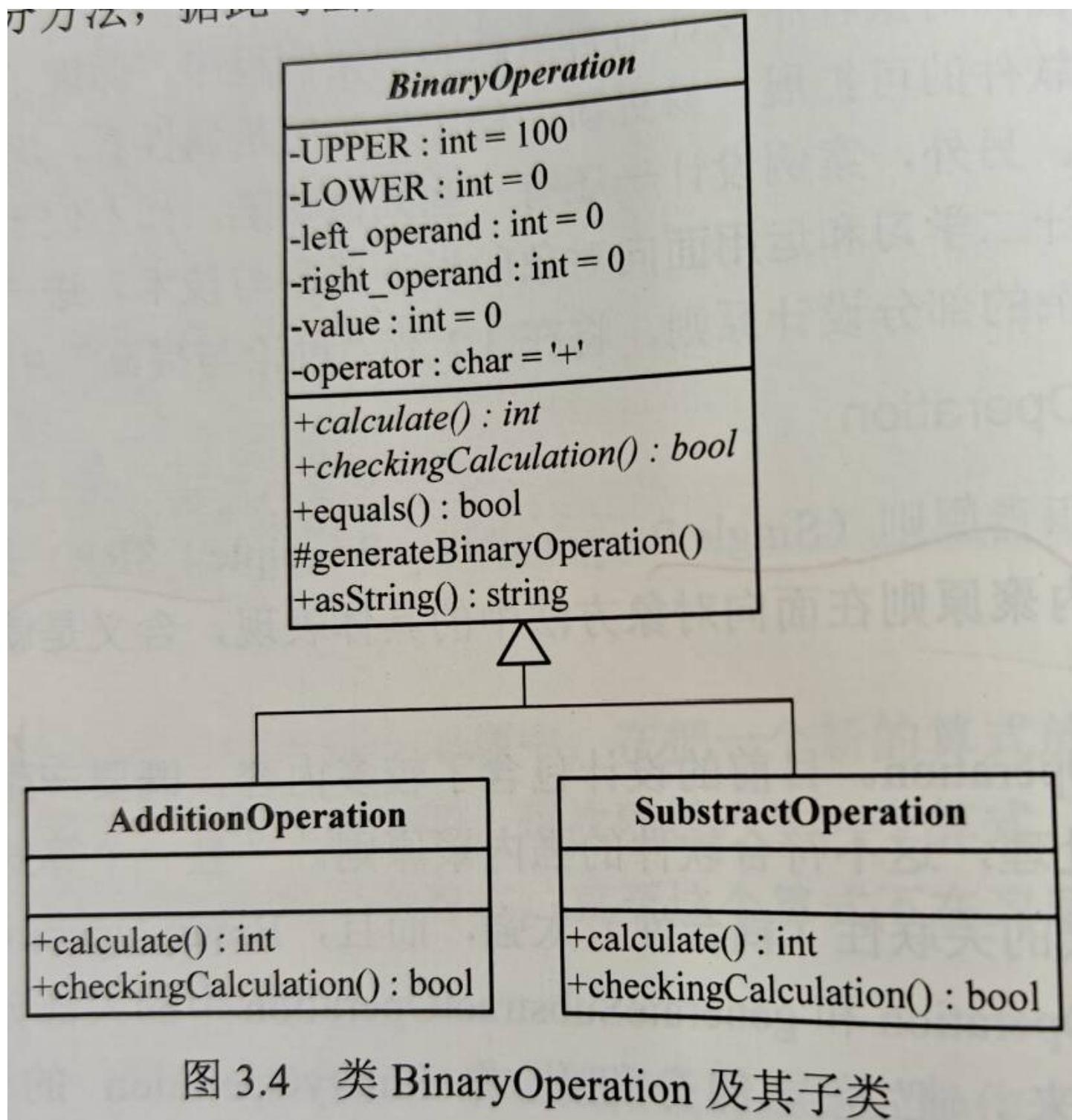


图 3.3 一个 UML 类关系图

类之间的关联用一根线表示，可以包括每个关联类的角色名、数目、方向和约束。泛化关系是一端带空心三角形的连线，绘制时从子类到父类，空心三角形一端是父类。有两种表示“整体-部分”的特殊关联——聚合和组合。聚合用来描述一个元素（整体）包含另外的元素（部分），部分可以脱离整体以一个独立个体存在。组合是一种语义更强的聚合，表示部分组成整体，不可分割，整体消失，部分也跟着消失，部分不能脱离整体而独立存在。在 UML 符号中，聚合的整体端用空心菱形表示，组合的整体端用实心菱形表示。图 3.3 的解释：机械工程师、电气工程师和软件工程师都属于员工，组合的整体端用实心菱形表示。图 3.3 的解释：机械工程师、电气工程师和软件工程师都属于员工，零件还可以（递归）包含零件。



JUnit基本信息、工作原理和基本注释

JUnit是Java语言的测试框架，实质上就是包含测试用例、测试执行、测试记录等的Java类和接口，用于对程序中的最小可测试单元进行自动化测试。

- **工作流程：** 编写测试类->编写测试方法->JUnit执行测试->使用断言判断结果->输出测试结果
- **工作原理：** 通过注解识别测试方法，并通过断言判断测试结果
 - 测试类中定义测试方法
 - JUnit测试运行器(Runner)自动识别测试方法
 - 按一定顺序执行测试方法

- 使用断言比较实际结果与期望结果
- 输出测试是否通过的结果

• 基本注释

- `@Test` 标识测试方法

代码块

```
1  @Test(expected = ArithmeticException.class)
2  public void testDivideByZero() {
3      int result = 10 / 0;
4  }某个测试方法在特定情况下是否会抛出指定异常,抛出该异常时测试通过
5
6  @Test(timeout = 1000)
7  public void testTimeout() {
8      while (true) {
9          // 模拟死循环
10     }
11 }限制测试方法的最大执行时间,防止死循环或性能问题
```

- `@Before` 在**每个测试方法执行前**执行,用于初始化

代码块

```
1  @Before
2  public void setUp() {
3      calculator = new Calculator();
4  }
```

- `@After` 在**每个测试方法执行后**执行,用于资源释放
- `@Ignore` 忽略某个测试方法。
- `@RunWith` 指定测试类的运行器,决定JUnit如何执行测试类

调试和测试的异同点

- **测试**是通过执行程序来发现软件中存在缺陷的活动,其目的是发现错误。不修改代码
- **调试**是在发现错误之后,定位、分析并修改程序中错误的过程
- **主要区别:** 目的:测试的目的是发现错误,调试的目的是定位并修复错误;是否修改程序:测试不修改程序,而调试修改程序;顺序:测试在前,调试在后,没有错误则不调试
- **共同点:** 目的都是提高软件质量;都围绕程序错误展开;都发生在软件构造阶段

合成与继承来实现复用的区别

- **继承**是通过建立类之间的“is-a（是一个）”关系，实现代码复用的一种机制。
 - 子类自动拥有父类的属性和方法
 - 静态
 - 属于**白盒复用**(复用者需要了解被复用组件的内部实现细节)，类之间耦合度较高
- **合成**是通过在一个类中包含另一个类的对象（has-a 关系），实现功能复用的一种机制。
 - 通过对象协作完成行为
 - 动态
 - 属于**黑盒复用**(复用者只依赖被复用组件的对外接口，而不关心其内部实现)，具有更好的封装性和灵活性
- 继承通过建立 is-a 关系实现代码复用，属于**白盒复用**，类之间耦合度较高；合成通过对象包含建立 has-a 关系实现复用，属于**黑盒复用**，具有更好的封装性和灵活性。面向对象设计中通常遵循“优先使用合成而非继承”的原则。
- 为什么说“优先使用合成而非继承”？
 - 继承的问题：子类强依赖父类实现；父类变化容易影响子类；易形成复杂的继承层次（类爆炸）；破坏封装性（子类了解父类细节）
 - 合成的优势：保持类的独立性；提高系统灵活性；更易扩展和维护；符合开闭原则和单一职责原则

第四章

序列化与反序列化的概念、用途、意义、含义

- **序列化**是将对象的状态信息(成员变量)转换为可以存储或传输的形式的过程 对象->字节序列
- **反序列化**是将字节序列重新还原为对象的过程
- 序列化使对象可以脱离 JVM 的内存环境而存在，反序列化使对象重新回到内存中继续使用。
- **为什么要序列化？**
 - 对象持久化(存储)：将对象保存到文件或数据库，程序结束后仍可恢复对象状态
 - 网络传输 / 分布式系统：对象不能直接在网络上传输，必须转换为字节流
 - 程序状态恢复：程序异常或重启后恢复现场，提高系统健壮性
- **序列化的意义？**
 - 提高系统可扩展性：支持分布式计算，支持跨进程通信
 - 提高系统可靠性：支持数据持久化，支持状态恢复
 - 提高软件复用性：对象可在不同系统间复用

防御性编程

- **基本思想：**程序员要预计其他程序员的过错、无效的输入，甚至有害的数据及使用者的过失，采取适当措施保护自己的程序正常运行
- 保护程序不被无效输入破坏的**基本原则：**检查每个输入参数的数据。特别要检查所有从程序外部源进入程序的数据
- **处理错误**
 - 继续运行程序，返回中性无害的数据
 - 用最接近的有效数据替换无效数据
 - 在日志中记录警告信息并继续运行程序
 - 调用错误处理程序或对象
 - 屏幕显示错误信息
 - 尽可能在局部处理错误
 - 返回一个错误编码，让特定程序处理这个错误
- **使用异常**
 - 终止模式，错误非常关键，一旦发生异常，立即终止当前操作
 - 恢复模式，异常发生后，在异常处理程序中修复问题，然后程序从异常发生点继续执行

断言

- **断言**是让程序在运行过程中自我检查的代码。若断言为真，则意味着程序如期望般正常
 - 前置断言：代码执行之前必须具备的特性
 - 后置断言：代码执行之后必须具备的特性
 - 不变断言：代码执行前后不能变化的特性
- `assertEquals([String message], expected, actual)`
比较两个基本类型或对象是否相等。`expected` 和 `actual` 是原始类型数值或者必须为实现比较而具有 `equal` 方法，字符串类型的消息 `message` 可选，语法方面用方括号表示。
- `assertFalse([String message],boolean condition)`
对布尔值求值，看它是否为“假”。
- `assertTrue([String message],boolean condition)`
对布尔值求值，看它是否为“真”。
- `assertNull([String message],java.lang.Object object)`
检查对象是否为“空”。
- `assertNotNull([String message],java.lang.Object object)`
检查对象是否不为“空”；
- `assertSame([String message] expected actual)`

assertSame([String message],expected,actual)

检查两个对象是否为同一实例。

● assertNotSame([String message],expected,actual)

检查两个对象是否不为同一实例。

● fail([String message])

使测试立即失败。这种断言通常被用于注解某个不应该到达的分支(例如, 在一个预期发生的异常之后) 或者系统建立测试类模板时对测试方法的初始化。

例如, 图 2 显示了运行初始化 BinaryOperationTest 的情形:

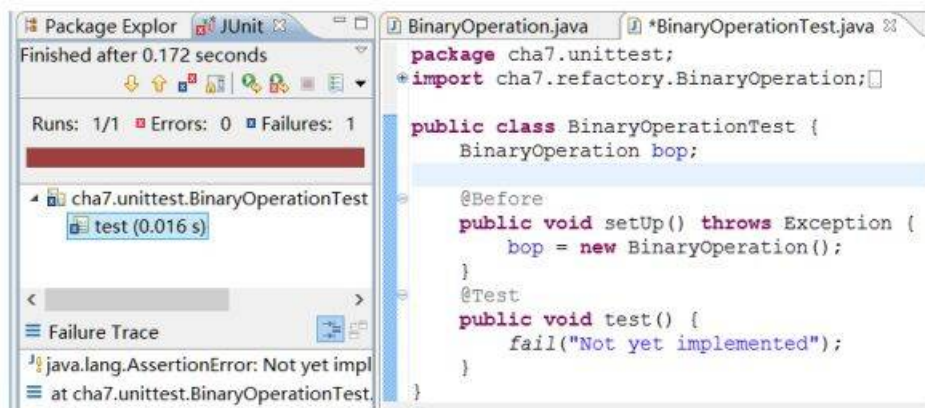


图 2 还没有编写测试代码时显示的测试失败

通过在 BinaryOperation 中添加一个算式比较方法 boolean equals(BinaryOperation), 并在测试类中增加 4 个测试, 看看断言 assertEquals、assertTrue 和 assertNull 的使用。测试代码及运行结果如下:

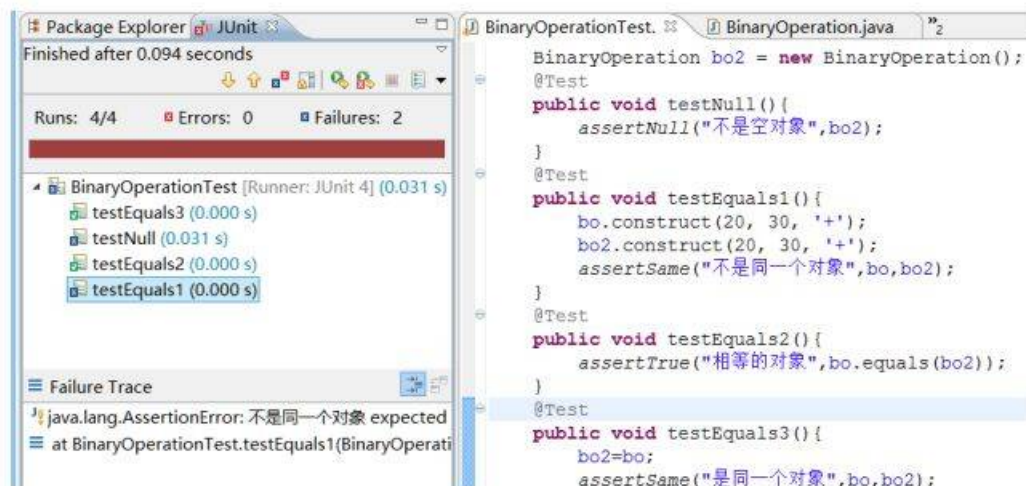


图 3 使用 assertNull、assertSame 和 assertTrue 测试断言的例子

JUnit 作为一个框架, 具有良好的扩展性。一些针对断言的扩展已经得到广泛的使用。

表驱动编程

- **表驱动编程**是一种将程序中的控制逻辑和决策规则以数据表的形式表示, 并通过查表来驱动程序执行的编程方法。
- **表项的查询**
 - 直接访问 下标 表中的项表示一个结果或动作

- 阶梯访问 表中的项表示一个数据范围而非单个数据，而且数据的排列按照一定的顺序 优点是容易处理不规则的数据
- 索引访问 为查询的数据建立一个检索表，用索引数据在索引表中得到关键字。然后用该值在另一表中检索感兴趣的主数据 优点是节省空间和操作检索表项要比操作主表项简单

逻辑覆盖测试和路径覆盖测试

- **判定覆盖(分支覆盖)**：程序中每个判断的真假分支至少执行一次
 - 一般而言，判定覆盖比语句覆盖要强
- **条件覆盖**：每个简单布尔条件的所有可能的值都至少满足一次
- **路径覆盖**：程序中所有可能的执行路径至少被执行一次

参数化测试

- **参数化测试**用于使用多组不同参数反复执行同一个测试方法，以减少重复代码，提高测试效率。
- 原理是将**测试逻辑**和**测试数据**分离
- 参数化测试用法
 - 指定运行器 `@RunWith(Parameterized.class)`
 - 定义参数集合

`@Parameters` 是 JUnit4 中用于参数化测试的注解，用来提供一组测试数据，使同一个测试方法可以在不同参数下被重复执行。

代码块

```
1  @Parameters
2  public static Collection<Object[]> data() {
3      return Arrays.asList(new Object[][] {
4          {1, 2, 3},
5          {2, 3, 5},
6          {0, 0, 0}
7      });
8  }
```

- 定义成员变量接收参数

代码块

```
1  private int a;
2  private int b;
3  private int expected;
```

- 构造函数接收参数

代码块

```
1 public CalculatorTest(int a, int b, int expected) {
2     this.a = a;
3     this.b = b;
4     this.expected = expected;
5 }
```

- 编写测试方法

代码块

```
1 @Test
2 public void testAdd() {
3     assertEquals(expected, add(a, b));
4 }
```

- 参数化测试的优点：避免重复编写测试代码；提高测试覆盖率；提高测试的可维护性

可以把参数化测试理解成测试一个数据集合。编写参数化测试的代码需要 6 个基本步骤。

(1) 为参数化测试编写一个独立的测试类，它不能与其他测试公用同一个类。本例新建了一个测试类 `BinaryOperationGroupTest`。

(2) 用语句 `@RunWith(Parameterized.class)` 为测试类指定 `org.junit.runners.Parameterized` 作为特殊的测试运行器，注解必须放在测试类的前面。

(3) 为测试类声明若干变量，分别用于存放期望值和测试数据。本例定义了三个参数和一个期望值。

(4) 为测试类声明一个带有参数的公共构造函数，为 (3) 中声明的变量赋值。

(5) 为测试类声明一个使用注解 `@Parameters` 修饰的、返回值为 `java.util.Collection` 的公共静态方法。在该方法中列出测试用例，即测试数据和期望值。该方法没有参数，名字没有特殊要求（本例是 `data`）。测试用例是一个二维数组，其中数据的一部分是与测试类的变量相同的数据，另一部分是预期结果。测试数据及预期结果的个数和顺序要与 (4) 的构造函数中的参数完全一致。

(6) 编写测试方法，使用定义的测试用例作为参数进行测试。

第五章

交互设计原则

- 3条基本原则

- 学习性。用户能够快速学习和掌握操作
- 灵活性。允许用户以不同方式完成任务

- 健壮性。系统能容忍用户操作错误，并提供纠正方式

● 8条黄金规则

- 尽量保持一致
- 满足普遍可用性
- 提供信息反馈。对用户的每个动作都要提供反馈
- 设计对话框显示结束信息
- 预防并处理错误
- 允许撤销操作
- 支持内部控制点
- 减轻短时记忆负担

● 菜单设计的一般指导原则

1. 按照系统的功能来组织菜单。菜单的宽度、深度和层次结构。逻辑相似的有联系的选项安排在一起；不同组之间要有分隔标志。
2. 选用广而浅的菜单结构。深度不易超过4层，每级菜单项数最好7到9个。
3. 菜单标题简短，含义明确，大致能解释菜单的作用。可以把第一级菜单设为主菜单，其中各项反应系统的基本功能和程序框架。
4. 合理命名各菜单项名称。使用熟悉的菜单项名，使用前后一致的和精确的措词，将关键词放在左边，且各选项彼此不同。菜单项名应体现所能完成的功能，使用语气亲切、含义明确、易于理解、简短明了的单词或动宾短语为菜单项名。例如：文件、字体、……、保存、关闭、拷贝、删除记录等等。
5. 根据菜单选项的含义进行分组，每组内选项要按一定的规则排序。菜单项的安排有利于提高菜单选取速度，可根据使用频度、数字顺序、字母顺序、功能逻辑顺序来组织安排菜单项顺序。
6. 依据使用频度排序：将经常使用的菜单项排在前面。
7. 依据功能逻辑顺序排序：如对文件的操作按新建、打开、读、写、保存、关闭顺序排序。
8. 依据使用顺序排序：显示用的排在前面。
9. 为菜单项提供多于一种的选择途径。常用选项要设置快捷键，允许超前键入，超前跳转或其他捷径跳转到上层菜单或主菜单，以适应不同水平的用户。
10. 要保持语法、布局、用词前后一致。一致性原则在界面设计中是强调最多的，因为它最容易被违反，同时也最容易修改和避免。例如，在菜单和联机帮助中必须使用相同的术语、对话框必须具有相同的风格。
11. 应该对菜单选择和点击设置反馈标记。例如，移动光标进行菜单选取时，光标所在的菜单项应该提供高亮度或反视屏显示，但此时并未选定，表示光标位于菜单的选项，当用户确认后，要有明确的操作来选取菜单项，如鼠标单击或按回车键。对选中的菜单也应该给出明确的反馈标记，例如，在选项前面加√等标记，对当前不可用的菜单项也应给出表示，如用灰色显示。
12. 对与正在进行的操作无关的菜单项加上禁用标记或干脆不显示，不过要慎用动态加载方式(即只显示需要的菜单)，以免迷惑用户。例如，Word中如果没有选择内容，则“剪切”和“复制”命令就不可用，此时就设为禁用状态，但能显示出来，如果不显示，用户会以为没有剪切和复制功能。
13. 菜单前的图标不宜太大，与文字保持一致最好。
14. 主菜单的宽度要接近，字数不应多于4个，每个菜单的字数相同是最好的，主菜单项目数量不应太多，至少在最大化时为单排布置，两排布置既难看又难用。
15. 要考虑联机帮助、新颖选择机制、响应时间、显示速率和屏幕尺寸。对于多数不熟练用户来说，联机帮助的作用非常大，如果用户做了一个不可接受的选择，应该在指定位置显示出错误信息。

快速原型法

- **快速原型**是原型制造的一种类型，重点是在开发过程的早期开发原型，尽早获得反馈和分析来支持开发过程
- **目的**是在获得用户交互基本需求说明的基础上，快速建立一个可以运行的软件，使用户及时运行程序，直观地看到软件交互的形式和使用效果，以便明确、补充和精细化交互的需求
- 功能选择->构造原型->运行和评价原型->修改和完善原型
 - 最终系统是软件需求全部功能的视线，而原型只实现所选择的部分功能
 - 最终系统对每个软件需求都要真实地实现，而原型仅仅是为了试验和演示用的，部分功能需求可以忽略或者模拟实现
- 优点：早期发现需求不明确问题；提高用户参与感；减少后期返工；快速验证系统可行性

基本的静态测试

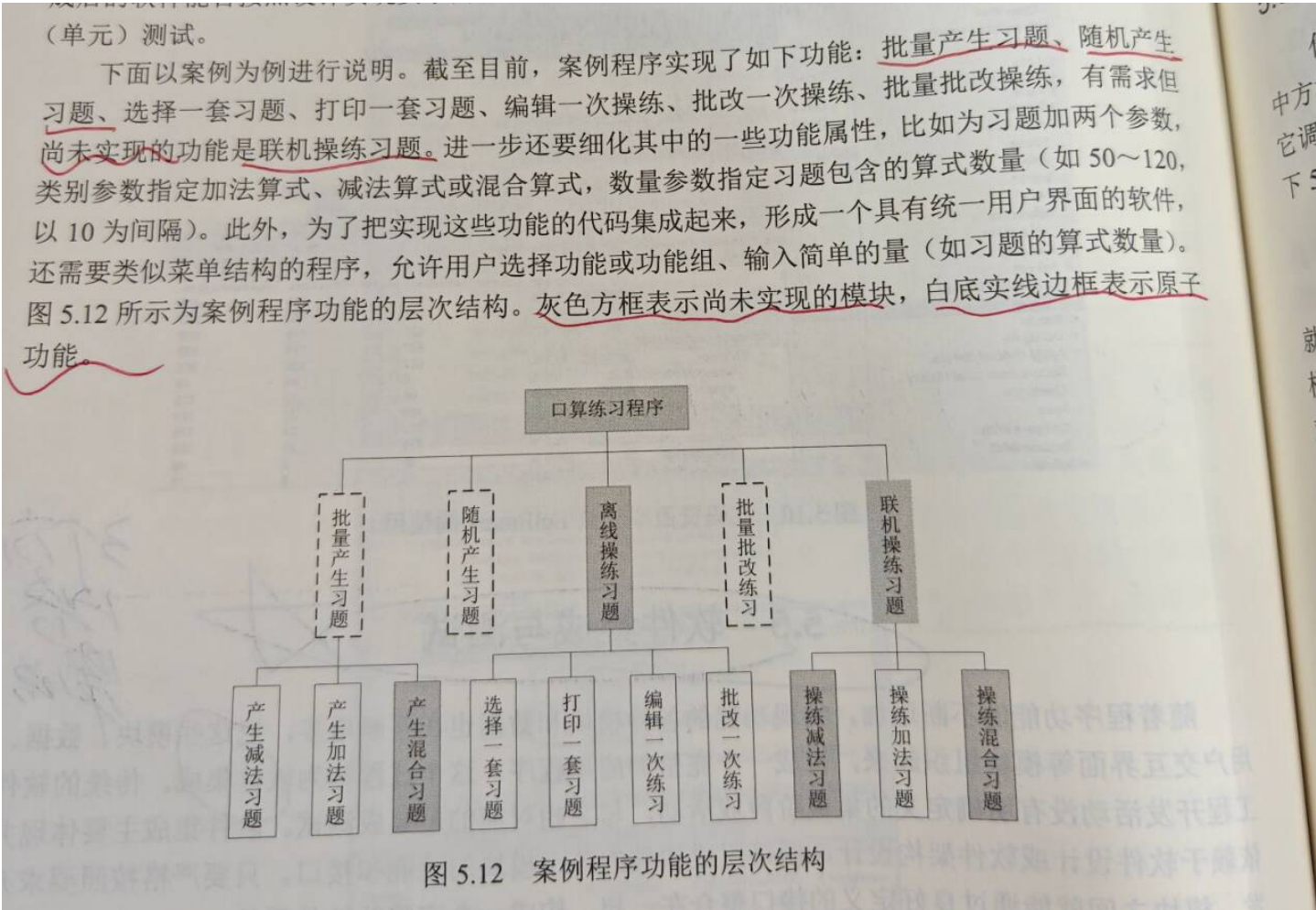
- 一种不运行程序，通过检查代码、文档或设计模型来发现缺陷的测试方法。

- **桌面检查** 无我编程：技术团队通过同级评审的方式来发现软件中的缺陷
- **代码走查** 至少两人
- **正式审查** 结构化、正式的评审过程
- **同行评审**

表 5.1 代码走查、正式审查和同行评审的异同点比较

角色/职责	正式审查	同行评审	代码走查
主持者	评审组长	评审组长或作者	作者
材料陈述者	评审者	评审组长	作者
记录员	是	是	可能
专门的评审角色	是	是	否
检查表	是	是	否
问题跟踪和分析	是	可能	否
产品评估	是	是	否
计划	有	有	是
准备	有	有	无
会议	有	有	有
修正	有	有	有
确认	有	有	无

程序层次结构图和集成策略



- **集成策略**：在集成测试阶段，将各个模块按一定顺序和方法组合成系统并进行测试的策略

- **一次性集成**
 - 首先完成模块的开发，然后连接在一起，构成最终的软件
- **增式集成**
 - 以一个基本模块或主模块为基础，每次增加一个或几个测试过的模块，在确保集成程序正确后继续添加模块，直至所有模块合在一起构成完整的软件
- **自顶向下集成**
 - 集成从根部的控制模块开始，以控制层次的顺序逐次增加模块，直至构成完整程序的增式集成方式
 - 优点：在继承的任何时候都有一个可以运行的程序
 - 缺点：需要大量桩模块；桩模块的存在使得集成程序难以得到充分的测试
- **自底向上集成**
 - 选择独立性最大或者对其他模块依赖性最小的模块，从程序结构图的底部开始，逐次增加控制模块，直至完成整个程序
 - 优点：底层模块的测试与继承可以独立；不需要桩模块
 - 缺点：驱动模块的开发成本一般较高；难以充分测试高层模块的可操作性
- **基于集成**
 - 选择一个程序的基干模块，将自顶向下与自底向上的集成方式结合起来，逐步集成其他模块直至实现完整程序策略
 - 优点：减少了自顶向下与自底向上集成的不利因素；集成过程总有一个可运行的程序；适合大型、复杂软件的集成
 - 缺点：需要仔细分析系统结构和模块间的依赖性，同时要分析用户需求的优先性；有时需要开发桩模块和驱动模块，工程量较大；局部采用了一次性集成，接口测试可能不充分

持续集成的概念

是敏捷开发中一项基本的软件开发实践。项目成员频繁地集成他们的工作——通常是每人每天的代码至少集成一次，使得每天都有多次集成的代码。每次集成都自动构建验证，可尽快发现错误

核心包括自动化构建、自动化测试及版本控制中心等工具

第六章

重构的概念、作用、基本操作、目的

- **代码重构**就是在不改变软件外部行为的前提下改善它的内部结构
- 重构既不修正错误，也不增加新的功能，主要作用在于改进代码质量，提高代码的可读性，或者改变代码内部结构与设计，使其更容易维护

- 代码重构主要有三个时机：给程序增量地添加新功能时、定位程序错误时和评审代码时
- 代码重构的主要原因：要改进软件的设计、要使软件更容易理解、为了方便查找错误
- **基本操作**：
 - **与方法（函数）相关的重构**
 - gz. 提炼方法 将一段功能单一的代码提取成独立方法
 - ha. 内联方法 将过于简单的方法合并回调用处
 - hb. 重命名方法 用更准确的名字表达方法意图
 - **与变量相关的重构**
 - d. 重命名变量 使用有意义的变量名代替模糊命名
 - e. 引入解释性变量 用变量代替复杂表达式
 - f. 消除临时变量 减少不必要的中间变量
 - **与类相关的重构**
 - a. 提炼类 将职责过多的类拆分成多个类
 - b. 合并类 将功能过少的类合并到相关类中
 - c. 重命名类 使类名准确反映其职责
 - **与接口和继承相关的重构**
 - j. 提炼接口 抽取公共行为形成接口
 - k. 用组合代替继承 减少继承带来的耦合
 - **与条件逻辑相关的重构**
 - a. 分解条件表达式 将复杂条件拆分成清晰的判断
 - b. 用多态替代条件 用子类行为代替大量 `if-else`

软件交付、虚拟机

- 软件开发和软件维护这两个阶段之间的开发活动统称为**软件交付**，目的是让最终用户使用开发的软件
- **基本活动**包括构建、打包、发布、安装和部署
- **软件环境**可以分为开发环境、运行环境和测试环境
- 最简单的软件交付活动是**编译**和**链接**，然后把应用代码**构建**成可直接运行的代码
- 软件交付的首要工作是把构造的程序从开发环境中分离出来并打包

- **虚拟机**是一种在计算机系统之上通过软件方式模拟出来的、具有完整计算机运行环境的逻辑计算机。
它为程序提供一个与具体硬件平台无关的运行环境，使程序可以在不同硬件和操作系统上以一致的方式运行。
- **软件编译后**并不是运行在源代码层面，而是运行在相应的运行环境中。对于编译为机器码的程序，运行在操作系统之上；对于编译为中间代码的程序，则运行在虚拟机中，由虚拟机再与操作系统和硬件交互。

测试层次

- **单元测试**
 - 对组成程序基本单元所进行的测试(函数、方法、类、模块)
 - 由开发人员完成
- **集成测试**
 - 对两个或两个以上相互关联的程序单元的测试
 - **重点是检测**程序模块的接口、模块间的交互及开发的软件与外部系统的交互
 - 主要由参与集成模块的开发小组完成，也可以由专职的测试人员完成
- **系统测试**
 - 对整个软件(及使用环境)的测试
 - **重点是检测**软件是否满足了用户需求、完成的既定功能和任务。还检查其他非功能需求
 - 由独立于开发人员的专职测试人员完成
- **验收测试**
 - 确保软件准备交付，可以在用户环境让最终用户使用，实现软件的既定功能和任务
 - 参与者包括软件的所有干系人

测试驱动开发

- **测试驱动开发TDD**要求在编写某个功能代码前先编写测试代码，然后只编写使测试通过的功能代码，通过测试来推动整个软件的开发
- 有助于编写简洁可用和高质量的代码，加速开发过程
- 基本过程：
 - 编写一个测试，检测待测程序是否满足一个新增的功能
 - 运行测试——由于还没有代码，甚至可能都不能通过编译
 - 编写待测程序足够的代码，使其通过编译
 - 编写待测程序满足测试，直至测试通过

- 必要的话重构代码，用回归测试验证
- 重复上述步骤，直到更多的需求都编码实现并通过测试
- **TDD原理**是在开发功能代码前，先编写单元测试代码，确定需要编写什么应用代码
- 考虑的是使用者的需求
- **特点：**
 - 只开发满足需求的软件
 - 促使实现松耦合的设计
 - 尽早地避免、发现和修改错误
 - 与软件同步的文档
 - 为快乐工作提供了基础

第七章

GUI基本设计规范

- **图形用户界面（GUI）**设计应以用户为中心，保证界面的一致性、易学性、易用性和健壮性，提高人机交互效率，减少用户错误。

PC平台通用软件的GUI基本设计规范

- **界面合理。**合理的布局和颜色搭配
- **风格一致。**使用标准的组件、明确定义的术语，与用户的习惯认知和软件的应用领域一致。界面的信息表现方式也要前后一致
- **元素标准。**基本元素的使用要符合标准
 - 窗口与对话框的标准
 - 菜单与工具条的标准
 - 图标的标准
 - 基本组件的标准

MVC模式与观察者模式

- **观察者模式：**定义了对象之间的一对多依赖关系，当一个对象状态发生变化时，所有依赖于它的对象都会得到通知并自动更新
 - 观察者一方面接受状态变化的通知，另一方面还要做出相应的动作，即在一个类中整合了输入解析和业务逻辑两部分功能，不符合单一职责的设计原则
- **MVC模式：**是一种软件架构模式，将应用程序分为模型、视图和控制器三个部分，实现关注点分离，提高系统的可维护性和可扩展性

- **模型**表示业务数据和业务逻辑，负责数据的存储、处理和状态维护
- **视图**负责数据的显示，不包含业务逻辑
- **控制器**处理用户输入，协调模型与视图的交互
- **各部分关系：**
 - **View与Controller：** 用户通过 View 发起操作；View 将用户操作交给 Controller 处理
 - **Controller与Model：** Controller 根据用户操作调用 Model；Model 执行业务逻辑并更新状态
 - **Model与View：** Model 状态变化；通知 View 更新显示

MVC简述，各部分作用和关系、用法

“模型/视图/控制器”（MVC，Model/View/Controller）的机制首先在 1979 年面向对象语言 Smalltalk 中出现。一般认为现在的 MVC 模式是观察者模式等几种设计模式的综合演变，是目前包括 Java GUI 程序、Web 应用程序、移动应用等多类软件采用的开发 GUI 的模式。MVC 模式中有三个角色：Model（模型）用于处理应用程序的数据逻辑，View（视图），用于处理应用程序的数据显示，Controller（控制器）用于处理应用程序的用户交互。

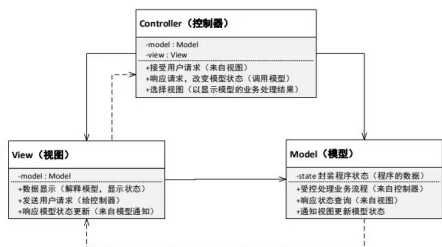


图 7.11 模式 MVC 的关系图

对比上一小节的观察者模式，被观察者就是 Model，观察者对应于 View，而缺少了由 View 实现的 Controller 功能。MVC 将 Controller 从 View 中分离出来，解除原先的紧耦合。

在 Swing 程序中，MVC 模式是这样体现的：Swing 的各个可视化组件都使用 MVC 模式来设计，Swing 组件一般由三个部分组成：

1. Swing 组件的 Model，是 MVC 的模型，包括 Swing 组件所有的状态（数据）。
 2. Swing 组件的外观，是 MVC 模式的视图，根据组件 Model 中的数据，展现相应的外观。
 3. Swing 组件上注册的侦听器，是 MVC 模式的控制器，捕捉并响应如按钮被点击等“事件”。
- 以上是简单的说明。我们介绍 Swing 的目的是以此为例讲解基于 MVC 模式的 GUI 软件构造方法。实际 Swing 组件类之间的关系较为复杂，不再进一步阐述。

- MVC 模式中，Model 与 View 之间通常通过观察者模式实现解耦(松耦合)。
 - View 作为 Observer
 - Model 作为 Subject
 - 当 Model 状态发生变化时，自动通知 View 更新界面

事件驱动编程

- **事件驱动编程**是一种以事件为中心的软件构造方式，程序的执行流程由外部或内部事件触发，通过事件的产生、分发和处理来完成相应的功能。
- 主要用于GUI和其他为响应用户的交互做出特定动作的应用程序
- 在事件驱动编程中，通常有一个监听事件的主循环，一旦检测到其中的一个事件，就触发一个函数调用、执行相应的动作
- 用户对于组件的动作或组件状态的改变都可以列入**事件**，发生动作或状态改变的这个组件就是**事件源**
- **焦点**是获取键盘或鼠标输入的能力

- 默认焦点应当处于用户经常选择的最先输入组件
- 对于具有连续多个文本输入组件的界面，应当从第一项输入组件开始，按照从左至右、从上到下的顺序依次切换

第八章

ER图

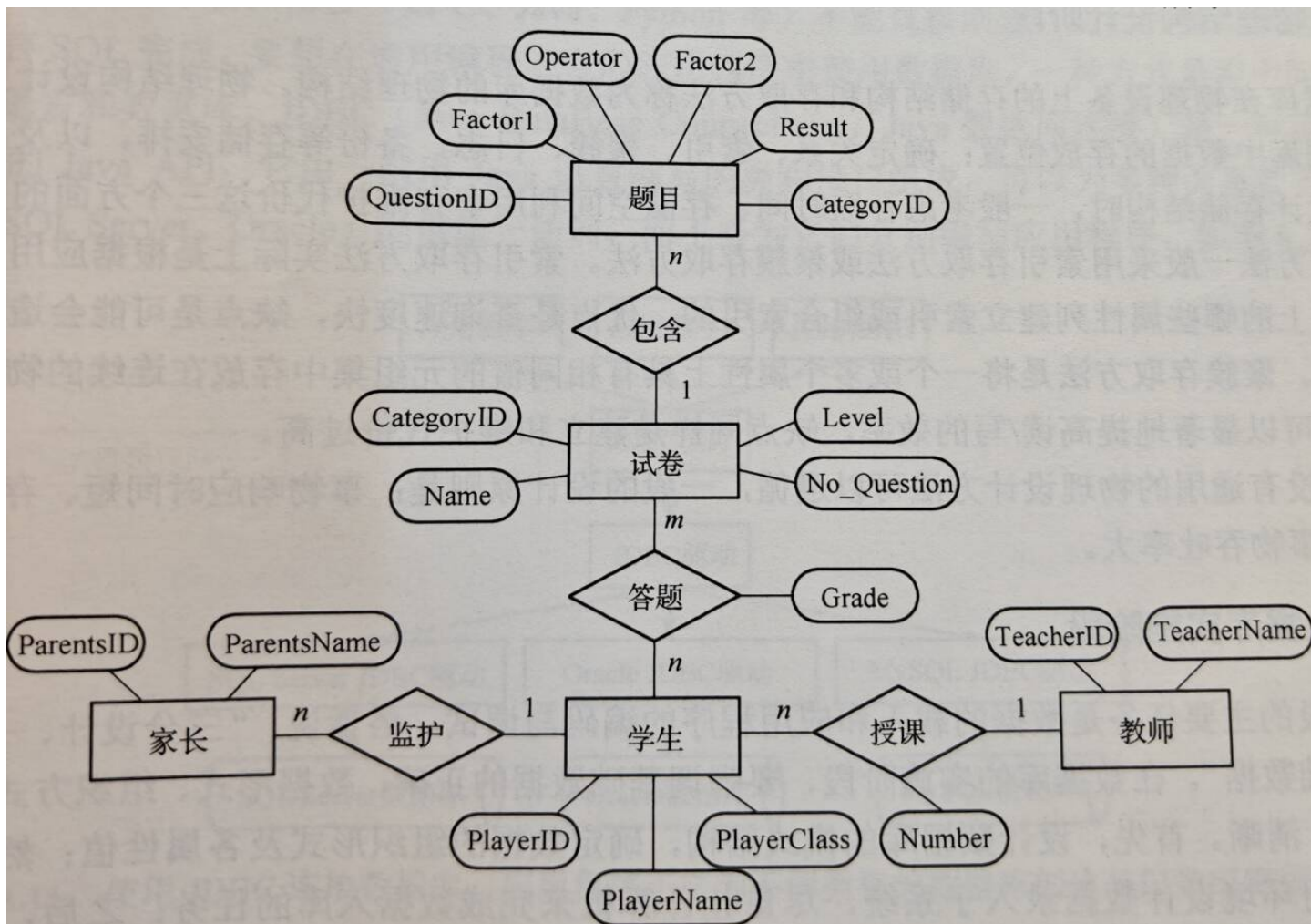


图 8.10 口算练习软件的 E-R 模型

JDBC工作原理

- **JDBC**是一种用于执行SQL语句的Java API，它由一组用Java语言编写的类和接口组成，可以为多种关系数据库提供统一访问，而不必为它们分别编写应用程序

通过统一的接口屏蔽不同数据库的差异，由具体数据库厂商提供驱动实现，从而实现数据库访问的可移植性

代码块

```

1  Java 应用程序
2      ↓
  
```



```
3  JDBC API (java.sql / javax.sql)
4      ↓
5  JDBC驱动
6      ↓
7  具体数据库驱动 (Driver)
8      ↓
9  数据库
```

- 在Java中使用JDBC操作数据库的主要步骤为
 - 与数据库建立连接
 - 向数据库发送SQL语句，需要java.sql包的支持
 - 处理SQL语句返回的结果
- **设计模式**
 - **桥联模式** JDBC 将数据库访问接口与具体数据库驱动实现分离
 - **工厂模式** 使用工厂模式创建数据库连接对象
根据数据库 URL 创建不同类型的 Connection
客户端无需关心具体实现类
 - **单例模式** 系统中只需要一个驱动管理器，统一管理数据库驱动
 - **模版方法模式** 数据库访问流程固定（连接 → 执行 → 处理结果）
 - **代理模式** 数据库连接池，事务管理

第九章

设计模式

- **设计模式**是对给定环境下反复出现问题的一个通用的、可复用的解决方案。它是可以在很多不同场合用以解决一类问题的一种描述或样板
- **模式**是程序员在设计一个软件或系统时解决共同问题最佳实践的正式描述
- **工厂方法模式**的实质是由一个工厂类根据传入的参数，动态地决定应该创建哪个产品类的实例。**核心思想**是：有一个专门的类来负责创建实例的过程，它把产品视为一系列类的集合，这些类是由某个抽象类接口或接口派生出来的一个类的树
- **适配器模式**

框架和设计模式的联系与区别

- 软构件通常是代码的复用，设计模式是设计的复用，框架则介于两者之间，既有代码复用，也含设计复用。

- 设计模式是对在某种环境中反复出现的问题及解决方案的描述，不能运行，比框架更抽象，框架通常用代码表示，可以全部或部分执行。
- 设计模式是比框架颗粒更小的设计元素，一个框架往往含有一种或多种设计模式。
- 框架针对某一特定的应用领域，同一个设计模式则可适用于各种应用。可以说，框架是软件，设计模式是软件的知识。

设计模式是设计层面的复用，不能直接运行；框架是介于设计模式和代码之间的可运行软件骨架，既包含设计复用，也包含代码复用。

设计模式比框架更抽象、粒度更小，一个框架通常由多种设计模式组成。

设计模式具有通用性，而框架面向特定应用领域。